

WEEK 4

ORDER MANAGEMENT SYSTEM

Scenario: An e-commerce system needs to store and manage customer orders efficiently. The system maintains order details such as customer, date, amount and status. It also stores the products, quantity and price related to each order. For this purpose, related tables and SQL queries are used to manage the data.

Table Creation

ORDERS Table

The ORDERS table is created to store the basic information about customer orders.

```
SQL> CREATE TABLE Orders(Order_ID VARCHAR(20),Customer_ID VARCHAR(20),Order_Date DATE,Total_Amount NUMBER(10,2),Order_Status VARCHAR(20));  
Table created.
```

ORDER_DETAILS Table

The ORDER_DETAILS table is created to store the product details associated with each order

```
SQL> CREATE TABLE Order_Details (Order_Detail_ID VARCHAR(20), Order_ID VARCHAR(20), Product_ID VARCHAR(10), Quantity NUMBER, Price NUMBER(10,2));  
Table created.
```

Relationship & Constraints

The **Customer** table is related to the **Orders** table through `Customer_ID`.
The **Orders** table is connected to the **Order_Details** table through `Order_ID`.
The **Order_Details** table is connected to the **Product** table through `Product_ID`. These relationships help to maintain the connection between customers, their orders and the products they purchase.

PRIMARY KEY

Primary keys were added to uniquely identify each record.

```
SQL> ALTER TABLE ORDERS ADD CONSTRAINT PK_Orders PRIMARY KEY (Order_ID);  
Table altered.  
  
SQL> ALTER TABLE ORDER_DETAILS ADD CONSTRAINT PK_OrderDetails PRIMARY KEY (Order_Detail_ID);  
Table altered.
```

FOREIGN KEY

Foreign Key constraints are used to establish relationships between the tables and maintain **referential integrity**.

```
SQL> ALTER TABLE ORDERS ADD CONSTRAINT FK_Orders_Customer FOREIGN KEY (Customer_ID) REFERENCES CUSTOMER(Customer_ID);  
Table altered.  
  
SQL> ALTER TABLE ORDER_DETAILS ADD CONSTRAINT FK_OD_Orders FOREIGN KEY (Order_ID) REFERENCES ORDERS(Order_ID);  
Table altered.  
  
SQL> ALTER TABLE ORDER_DETAILS ADD CONSTRAINT FK_OD_Product FOREIGN KEY (Product_ID) REFERENCES PRODUCT(Product_ID);  
Table altered.
```

NOT NULL

```
SQL> ALTER TABLE ORDERS MODIFY Order_Date NOT NULL;  
  
Table altered.
```

CHECK

Check constraints were added to prevent invalid values.

```
SQL> ALTER TABLE ORDERS ADD CONSTRAINT CHK_Total CHECK (Total_Amount >= 0);
Table altered.

SQL> ALTER TABLE ORDERS ADD CONSTRAINT CHK_Status CHECK (Order_Status IN ('Pending', 'Shipped', 'Delivered'));
Table altered.

SQL> ALTER TABLE ORDER_DETAILS ADD CONSTRAINT CHK_Qty CHECK (Quantity > 0);
Table altered.

SQL> ALTER TABLE ORDER_DETAILS ADD CONSTRAINT CHK_OD_Price CHECK (Price >= 0);
Table altered.
```

Insert values to Orders

```
SQL> INSERT INTO ORDERS VALUES('111',101,TO_DATE('2025-01-10','YYYY-MM-DD'),2500.00,'Delivered');
1 row created.

SQL> INSERT INTO ORDERS VALUES('112',102,TO_DATE('2025-02-15','YYYY-MM-DD'),307.00,'Shipped');
1 row created.
```

```
SQL> INSERT INTO ORDERS VALUES('113',103,TO_DATE('2025-03-20','YYYY-MM-DD'),4800.00,'Pending');
1 row created.

SQL> INSERT INTO ORDERS VALUES('114',104,TO_DATE('2025-04-05','YYYY-MM-DD'),400.00,'Pending');
1 row created.

SQL> INSERT INTO ORDERS VALUES('115',105,TO_DATE('2025-05-12','YYYY-MM-DD'),5071.00,'Delivered');
1 row created.
```

Display Orders Table

```
SQL> select * from orders;
```

ORDER_ID	CUSTOMER_ID	ORDER_DAT	TOTAL_AMOUNT	ORDER_STATUS
111	101	10-JAN-25	2500	Delivered
112	102	15-FEB-25	307	Shipped
113	103	20-MAR-25	4800	Pending
114	104	05-APR-25	400	Pending
115	105	12-MAY-25	5071	Delivered

Insert Values to Order Details:

```
SQL> INSERT INTO ORDER_DETAILS VALUES('OD101','111','P001',5,1000);  
1 row created.  
  
SQL> INSERT INTO ORDER_DETAILS VALUES('OD102','112','P002',10,2000);  
1 row created.  
  
SQL> INSERT INTO ORDER_DETAILS VALUES('OD103','113','P003',15,3000);  
1 row created.  
  
SQL> INSERT INTO ORDER_DETAILS VALUES('OD104','114','P004',20,4000);  
1 row created.
```

Display Order Details Table:

```
SQL> select * from order_details;
```

ORDER_DETAIL_ID	ORDER_ID	PRODUCT_ID	QUANTITY	PRICE
OD101	111	P001	5	1000
OD102	112	P002	10	2000
OD103	113	P003	15	3000
OD104	114	P004	20	4000

Update :

```
SQL> UPDATE CUSTOMER SET CUSTOMER_NAME='Arun Kumar Raj' WHERE CUSTOMER_ID=101;  
1 row updated.
```

Delete:

```
SQL> DELETE FROM CUSTOMER WHERE CUSTOMER_ID=105;  
1 row deleted.
```

Generate Customer Order History Report

Report 1: Customer Order History

Display:

- Customer name
- Order ID
- Order date
- Total Amount
- Status

```
SQL> SELECT C.CUSTOMER_NAME,O.ORDER_ID,O.ORDER_DATE,O.TOTAL_AMOUNT,O.ORDER_STATUS FROM CUSTOMER C JOIN ORDERS O ON C.CUSTOMER_ID=O.CUSTOMER_ID;
```

CUSTOMER_NAME	ORDER_ID	ORDER_DAT	TOTAL_AMOUNT	ORDER_STATUS
Arun Kumar	111	10-JAN-25	2500	Delivered
Priya Sharma	112	15-FEB-25	307	Shipped
Rahul Raj	113	20-MAR-25	4800	Pending
Divya Devi	114	05-APR-25	400	Pending
Karthik Raj	115	12-MAY-25	5071	Delivered

Report 2: Product wise Order Report

Find :

- Number of times each product was ordered.
- Total quantity sold

```
SQL> SELECT P.PRODUCT_ID,P.PRODUCT_NAME,COUNT(DISTINCT OD.ORDER_ID) AS TIMES_ORDERED FROM PRODUCT P JOIN ORDER_DETAILS OD ON P.PRODUCT_ID=OD.PRODUCT_ID GROUP BY P.PRODUCT_ID,P.PRODUCT_NAME ORDER BY P.PRODUCT_ID;
```

PRODUCT	PRODUCT_NAME	TIMES_ORDERED
P001	Ball Pen	1
P002	Gel Pen	1
P003	Pencil	1
P004	Pencil Box	1
P005	Spiral Notebook	1

```
SQL> SELECT P.PRODUCT_ID,P.PRODUCT_NAME,SUM(OD.QUANTITY) AS TOTAL_QUANTITY_SOLD FROM PRODUCT P JOIN ORDER_DETAILS OD ON P.PRODUCT_ID=OD.PRODUCT_ID GROUP BY P.PRODUCT_ID,P.PRODUCT_NAME ORDER BY P.PRODUCT_ID;
```

PRODUCT	PRODUCT_NAME	TOTAL_QUANTITY_SOLD
P001	Ball Pen	5
P002	Gel Pen	10
P003	Pencil	15
P004	Pencil Box	20
P005	Spiral Notebook	14

Report 3: Customer Purchase Analysis

Find:

- Customers with maximum orders.
- Total spending by each customer.

- Average order value.

```
SQL> SELECT C.CUSTOMER_ID,C.CUSTOMER_NAME,COUNT(O.ORDER_ID) AS TOTAL_ORDERS FROM CUSTOMER C JOIN ORDERS O ON C.CUSTOMER_ID=O.CUSTOMER_ID GROUP BY C.CUSTOMER_ID,C.CUSTOMER_NAME ORDER BY C.CUSTOMER_ID;
```

CUSTOMER_ID	CUSTOMER_NAME	TOTAL_ORDERS
101	Arun Kumar	1
102	Priya Sharma	1
103	Rahul Raj	1
104	Divya Devi	1
105	Karthik Raj	1

```
SQL> SELECT C.CUSTOMER_ID,C.CUSTOMER_NAME,SUM(O.TOTAL_AMOUNT) AS TOTAL_SPENDING FROM CUSTOMER C JOIN ORDERS O ON C.CUSTOMER_ID=O.CUSTOMER_ID GROUP BY C.CUSTOMER_ID,C.CUSTOMER_NAME ORDER BY C.CUSTOMER_ID;
```

CUSTOMER_ID	CUSTOMER_NAME	TOTAL_SPENDING
101	Arun Kumar	2500
102	Priya Sharma	307
103	Rahul Raj	4800
104	Divya Devi	400
105	Karthik Raj	5071

```
SQL> SELECT C.CUSTOMER_ID,C.CUSTOMER_NAME,ROUND(AVG(O.TOTAL_AMOUNT),2) AS AVERAGE_ORDER_VALUE FROM CUSTOMER C JOIN ORDERS O ON C.CUSTOMER_ID=O.CUSTOMER_ID GROUP BY C.CUSTOMER_ID,C.CUSTOMER_NAME ORDER BY C.CUSTOMER_ID;
```

CUSTOMER_ID	CUSTOMER_NAME	AVERAGE_ORDER_VALUE
101	Arun Kumar	2500.00
102	Priya Sharma	307.00
103	Rahul Raj	4800.00
104	Divya Devi	400.00
105	Karthik Raj	5071.00